
Package Identity and the Limits of SBOM Vulnerability Matching

Evidence from 118 conda and pixi workspaces

Kevin Mills

September 2026

ABSTRACT

A software bill of materials is only useful to the extent that the identifiers inside it can be matched against something. This paper examines a structural failure in that matching for software built with conda, a package ecosystem in wide use across scientific computing, data science and machine learning. Conda packages are conventionally identified by `pkg:conda` package URLs, and no major vulnerability database indexes that namespace. A document can therefore be complete, schema-valid, and satisfy every published minimum-elements requirement while producing an empty vulnerability report. We measured this on 118 public conda and pixi workspaces containing 15,722 packages. Identifying conda-distributed Python packages by their PyPI identity rather than their conda identity revealed 1,274 advisories that were invisible otherwise, a 37.1 percent increase, including 32 additional critical and 600 additional high-severity findings. Of the 96 workspaces carrying at least one advisory, 45 reported none at all under conventional identification. No advisory was lost by the change. We describe the mechanism, quantify the exposure, set out what a correct implementation requires, and discuss the consequences for organizations now subject to SBOM obligations.

Introduction

For most of their history, software bills of materials were a thing you produced because someone asked. That has changed. In the United States, the Cybersecurity and Infrastructure Security Agency published a revised set of minimum elements on 29 July 2026, replacing the 2021 NTIA baseline with seventeen required data fields and six process practices. Medical device submissions to the FDA have required a bill of materials under section 524B of the FD&C Act since 2023. The European Union's Cyber Resilience Act carries its own documentation duties, with reporting obligations under Article 14 having begun on 11 September 2026. Germany's BSI publishes a technical requirement, TR-03183-2, that specifies field-level content.

The obligation is now real, and so is the temptation to satisfy it on paper. A bill of materials is a structured document. Producing one that validates against its schema is not difficult. Producing one that is *useful* requires something the schema cannot check: that the identifiers inside it refer to things other systems can recognize.

This paper is about a case where that fails silently, at scale, in an ecosystem with a large and growing installed base.

Background: identity is the whole job

Strip away the metadata and a bill of materials does one thing. It says: this software contains these components. Every downstream use depends on turning each of those component entries into a lookup against some other corpus, whether a vulnerability database, a license index, or an export-control list.

The industry's answer to how components are named is the package URL, or purl. A purl is a compact string with a type, a namespace, a name and a version:

```
pkg:pypi/pillow@10.2.0
pkg:npm/express@4.18.2
pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1
```

The type is the first segment and it does the heavy lifting. It tells a consumer which registry the name belongs to, and therefore which corpus to query. This works well when the type corresponds to an ecosystem that vulnerability databases actually index. The Open Source Vulnerabilities database, the GitHub Advisory Database, and essentially every commercial scanner index `pkg:pypi`, `pkg:npm`, `pkg:maven`, `pkg:golang`, `pkg:cargo` and a handful of others.

They do not index `pkg:conda`.

The mechanism

Conda is a package manager and distribution system widely used in scientific computing, data science and machine learning. Its defining characteristic, and the reason for its adoption, is that it distributes more than Python packages. A conda environment contains compilers, shared libraries, CUDA runtimes, R packages and system-level dependencies alongside Python ones, all resolved together by a single solver.

pixi is a newer workspace-oriented package manager built on the same package format and channels, with a lockfile, named environments, and per-platform resolution.

The relevant consequence is this. When a Python package such as `pillow` is installed from conda-forge rather than PyPI, it is a conda package. The natural identifier for it, and the one tooling conventionally emits, is:

```
pkg:conda/pillow@10.2.0
```

That string is correct. It accurately describes where the package came from and how it was installed. It is also unmatchable. No advisory database has a `pkg:conda` namespace to query, so a scanner reading that identifier returns nothing. Not an error, not a warning, not an “unknown ecosystem” diagnostic. Nothing. The same package installed from PyPI, carrying `pkg:pypi/pillow@10.2.0`, returns its full advisory history.

The failure has three properties that make it unusually dangerous:

It is silent. An empty result set and a result set that could not be computed are indistinguishable to the consumer. The report renders as a clean bill of health.

It is invisible to validation. The document is schema-valid. It satisfies the CISA minimum elements, which require a unique identifier but do not and cannot require that the identifier be resolvable against any particular corpus. Every conformance check passes.

It scales with the thing you were trying to protect. The more of your stack you install through conda, which is the entire point of using conda, the larger the unmatched fraction becomes.

Method

We assembled a corpus of public workspaces and measured the difference that identity resolution makes to vulnerability matching.

Corpus. We queried the GitHub code search API for files named `pixi.lock`, collected the distinct repositories, and retrieved one lockfile per repository from its default branch. Of 297 distinct repositories found, 145 lockfiles were retrieved and parsed successfully, and 118 resolved for the `linux-64` platform and are reported here. Workspaces were not filtered for size, domain, activity or quality. The full repository list accompanies this paper.

Arms. Each workspace was processed twice, differing only in how conda packages were identified:

- *Naive.* Components carry the identifiers recorded in the lockfile. Conda packages are `pkg:conda`. PyPI packages installed as PyPI packages retain `pkg:pypi`. This models a tool that reads the lockfile and emits what it finds, which is the conventional behavior.
- *Mapped.* Conda packages that correspond to a PyPI project are additionally given their `pkg:pypi` identity, resolved through the conda-forge name mapping that the package managers themselves use, and that identity is the one offered for matching. The conda identity is retained as a secondary reference.

Both arms then queried the same vulnerability corpus, the Open Source Vulnerabilities database, on 30 September 2026, and cross-referenced CISA’s Known Exploited Vulnerabilities catalog.

Measurements. Per workspace we recorded the component count, the conda and PyPI split, the number of conda packages that acquired a PyPI identity, the set of distinct advisory

identifiers returned in each arm, and the severity of each.

Instrument. Measurements were taken with pixi-sbom 1.0.0, which implements both identification strategies as a command-line option. The choice of instrument does not affect the result, which is a property of the identifiers and the databases, not of any particular tool.

Findings

Composition

The corpus contained 15,722 packages across 118 workspaces. 14,120 of them, or 89.8 percent, were conda packages. This is the expected shape of a conda environment and it is worth stating plainly: in a typical workspace, the components a `pkg:conda` identifier makes unmatchable are the overwhelming majority of the document.

Of those 14,120 conda packages, 4,572 corresponded to a PyPI project and could be given a PyPI identity. That is 32.4 percent of the conda packages, and 29.1 percent of the corpus overall. The remainder are compilers, shared libraries, system tooling and non-Python language packages, for which no PyPI identity exists and none should be invented.

Advisories

	NAIVE	MAPPED	DIFFERENCE
Distinct advisories	2,158	3,432	+1,274
Critical	90	122	+32
High	1,043	1,643	+600
Medium	826	1,371	+545
Low	185	272	+87
Unspecified	14	24	+10

37.1 percent of the advisories affecting this corpus were invisible to a tool reading conventional identifiers.

The naive arm is not a strawman. It found 2,158 advisories, because a lockfile also records genuine PyPI packages and those match normally. The gap measured here is specifically the conda-distributed Python packages, and it is a gap in otherwise working tooling.

No advisory was lost. Across all 118 workspaces, the number of advisories present in the naive arm but absent in the mapped arm was zero. Resolving identity is strictly additive. This matters for the objection that mapping trades one kind of error for another: in this corpus it

did not, because the conda identity is retained as a secondary reference and nothing is discarded.

Distribution across workspaces

96 of 118 workspaces, or 81.4 percent, carried at least one advisory when components were correctly identified.

Of those 96 affected workspaces, 45 reported zero advisories under naive identification.

Nearly half of the projects that had a vulnerability problem looked entirely clean. Not under-reported. Clean.

Among affected workspaces the median count was 23.5 advisories and the maximum was 232.

Which packages

The newly visible advisories concentrated in exactly the packages one would expect to find in a scientific Python environment:

PACKAGE	WORKSPACES AFFECTED
pillow	259
tornado	204
mistune	116
jupyterlab	66
urllib3	58
jupyter_server	54
pip	43
gitpython	42
starlette	31
setuptools	30

These are not exotic components. They are the load-bearing libraries of the Jupyter and scientific Python stack, they are extremely well covered by advisory databases, and they were invisible because of how they were installed.

A negative result

Both arms surfaced the same five advisories in CISA's Known Exploited Vulnerabilities catalog. Identity resolution added none.

We report this because it runs against the direction of the rest of the findings. The KEV catalog is small and heavily weighted toward network-reachable enterprise software, so the overlap

with a scientific Python environment's dependency set is thin in either arm. The exposure demonstrated here is real but it is a general vulnerability-management exposure, not, on this evidence, a known-exploited one.

Consequences for compliance

The regulatory instruments now in force describe what a document must contain. They are largely silent on whether its contents can be resolved, which is the gap this paper is about.

CISA minimum elements. The July 2026 revision requires a unique identifier for each component. A `pkg:conda purl` is a unique identifier and satisfies the field. The document conforms while the identifier resolves against nothing.

NIST SP 800-218, the Secure Software Development Framework. Practice PS.3.2 calls for collecting and maintaining provenance data for the software and its components. An organization can maintain provenance accurately and still not know what is wrong with what it maintains.

PCI DSS 4.0. Requirement 6.3.2 obliges an entity to maintain an inventory of bespoke, custom and third-party software components to facilitate vulnerability management. The inventory here exists. The facilitation does not happen.

Financial-sector operational resilience. DORA in the European Union and NYDFS Part 500 in New York both require institutions to identify and manage third-party technology risk on an ongoing basis. Quantitative research environments in these institutions are a heavy conda constituency, and are precisely where this failure mode lives.

FDA section 524B. Device submissions must include a bill of materials. Scientific and imaging software in this category is commonly conda-based.

The pattern is consistent. Compliance regimes specify document contents because that is what a regulator can audit. They cannot easily specify that the contents be *useful*, and so an organization can be fully compliant and materially exposed at the same time. On this corpus, nearly half of the affected projects would pass an inventory audit and a vulnerability scan simultaneously, while carrying a median of 23 advisories.

What a correct implementation requires

The following are ecosystem-neutral. They apply to any tool producing bills of materials for environments that mix package ecosystems, which increasingly means any tool at all.

1. **Resolve identity to the ecosystem the consumer can query, not only the one the package came from.** A conda-distributed Python package has two true identities. The document should carry both, and the one offered for matching should be the one a database indexes.

2. **Retain the installation identity.** The conda identifier records real provenance: the channel, the build string, the platform. It answers questions the PyPI identifier cannot, including reproducibility and channel trust. Replacing it is a loss. Both belong in the document.
3. **Use the ecosystem's own name mapping.** conda-forge publishes a mapping from conda package names to PyPI project names, and the package managers themselves consume it. Hand-maintained mapping tables and name-similarity heuristics both drift. Neither is necessary.
4. **Do not invent identities that do not exist.** Two thirds of the conda packages in this corpus have no PyPI equivalent because they are compilers, shared libraries and non-Python packages. A tool that guesses at these produces false matches, which are worse than no matches because they consume triage effort.
5. **Distinguish “no findings” from “could not query”.** This is the single most valuable change available to tool authors. A consumer that knows an identifier was unmatchable can act on it. A consumer handed an empty list cannot. Emitting a count of unresolvable components would be a small change with a large effect.
6. **Report coverage as a first-class metric.** A bill of materials should be able to state what fraction of its components carry a queryable identity. That number, not the document's schema validity, is what predicts whether the scan means anything.

Recommendation 5 deserves emphasis. Every other item on this list is work for tool authors in one ecosystem. That one is a general principle, and the failure it describes, silently returning an empty result where an error was warranted, is not specific to conda or to bills of materials at all.

Related work and current tooling

Several general-purpose scanners can inspect a conda environment's installed files and identify Python distributions from their metadata, which sidesteps the purl question by not relying on lockfile identity. This works on a materialized environment and does not help when the artifact under analysis is a lockfile, which is the common case in continuous integration and in supply chain review, where the environment has not been built.

Tooling in the pixi and conda ecosystem is younger. The measurements here were taken with `pixi-sbom`, an open source tool that reads `pixi.lock` directly and implements the identity resolution described above, including the conda-forge mapping and the dual-identity representation. It emits CycloneDX 1.6 and 1.7 and SPDX 2.3 and 3.0.1. Its approach to this problem is one implementation of recommendations 1 through 4; it does not currently implement 5 or 6 either, which is the honest state of the art.

The authors of this paper maintain that tool. Readers should weigh the recommendations on their merits and note that the underlying finding, that `pkg:conda` identifiers do not resolve

against advisory databases, is verifiable in a few minutes against any conda environment and any scanner.

Limitations

Corpus selection. Public repositories containing a committed `pixi.lock` are not a random sample of conda-using software. pixi is newer than conda and skews toward recently started projects. Private and enterprise environments, where the compliance stakes are highest, are absent by construction.

Single platform. Only `linux-64` resolutions were measured. Workspaces locking other platforms were dropped rather than analyzed separately.

Single database. Advisory counts come from OSV. A commercial scanner with proprietary intelligence would return different absolute numbers. The relative gap is a property of purl namespaces and should hold, but we did not verify it against a commercial product.

Point in time. Advisory databases change daily. The counts describe 30 September 2026.

Severity accounting. Where an advisory carries several ratings we took the first. Severity totals are therefore indicative, and the advisory counts, which are exact, are the more reliable figure.

Unpatched advisories are not the same as exploitable ones. These counts say nothing about reachability. A vulnerability in an installed package that is never called is a lower priority than the count implies, which is what VEX exists to express. The finding is about what a scan can see, not about what an organization should drop everything to fix.

Conclusion

The gap described here is not a bug in any particular tool and not a deficiency in the purl specification, which is doing exactly what it was designed to do. It is an integration failure that falls between an ecosystem that names packages one way and a database industry that indexes them another, and it persists because every individual component of the system reports success.

Its practical form is that a large fraction of scientific, data science and machine learning software currently produces bills of materials that satisfy their regulatory purpose and answer no security question. In this corpus, 37 percent of advisories were unmatchable, and nearly half of the affected projects returned a clean report while carrying a median of 23 findings.

The remedy is well understood and cheap: resolve the identity, keep both, and say so when you cannot. The reason to write it down is that the current state is not visible to the people relying on it. A clean vulnerability report on a conda environment is not evidence of a secure environment. It is, more often than not, evidence that nothing was asked.

Reproducing this

The corpus repository list, the per-workspace measurements and the analysis script accompany this paper. The measurement itself is two commands per workspace:

```
pixi-sbom --lockfile pixi.lock --platform linux-64 \  
  --pypi-mapping lock --primary-purl conda \  
  --vulnerabilities osv --kev --output naive.json  
  
pixi-sbom --lockfile pixi.lock --platform linux-64 \  
  --pypi-mapping prefix --primary-purl pypi \  
  --vulnerabilities osv --kev --output mapped.json
```

The difference between the `vulnerabilities` arrays of those two documents is the subject of this paper.

pixi-sbom is open source under Apache 2.0 at github.com/millsks/pixi-sbom. Correspondence to the author.